

MIGRACIÓN DE PROYECTO JPA/HIBERNATE A SPRING DATA JPA CON SPRING BOOT

Esta actividad tiene como objetivo completar la migración de nuestro anterior proyecto del Instituto de JPA/Hibernate a Spring Data JPA con Spring Boot. Para ello, hemos hecho una clonación del proyecto donde se han realizado cambios tanto en la estructura como internamente en las clases. Se han añadido dependencias como Spring Web para la creación de aplicaciones web y APIs REST. También se ha añadido Spring Data JPA para trabajar de manera más sencilla con las bases de datos que nos va a generar de manera automática ciertas consultas. Y, Lombok, que es una librería de Java que nos va a evitar escribir código repetitivo y no ahorrará mucho tiempo.

1. Entidad elegida para pruebas de Spring Boot

Una vez instaladas las nuevas dependencias y estructurado el directorio del nuevo proyecto, nos vamos a centrar en una de las entidades para hacer las pruebas con Spring Boot. En este caso, se ha elegido la entidad Alumno para ver cómo queda estructurada la clase y sus clases derivadas. En cuanto a la clase Alumno principal no hay mucho cambio con respecto al proyecto de JPA/Hibernate.

```
import org.springframework.data.annotation.Data;  
  
@Data  
@Entity  
@Table(name = "alumnos")  
public class Alumno {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    @Column(name = "id_alumno")  
    private Integer idAlumno;  
  
    @Column(name = "nombre", nullable = false, length = 32)  
    private String nombre;  
  
    @Column(name = "apellidos", nullable = false, length = 64)  
    private String apellidos;  
  
    @Column(name = "fecha_nac", nullable = false)  
    private LocalDate fechaNac;  
}
```

La diferencia real que tiene es la notación que añadimos el inicio de `@Data` que hace que Lombok genere automáticamente los `@Getters`, `@Setters`... Todo lo demás se mantiene igual haciendo referencia la tabla, las columnas y declarando las variables privadas en referencia a los campos de la base de datos.

2. Métodos DAO sustituidos por métodos del repositorio

El siguiente paso una vez creada modificada la clase Alumno es crear una interfaz AlumnoRepository que será similar a AlumnosDAO con la diferencia de que la de AlumnoRepository va a heredar de JpaRepository sin nosotros tener que crear los métodos porque ya están en la librería de Spring Data JPA. De esta manera, una vez hecho el AlumnoRepository vamos a crear el AlumnoController para ahí sí definir los métodos de nuestra clase Alumno.

```
package com.example.demo.Repository;

import ...

public interface AlumnoRepository extends JpaRepository<Alumno, Integer> { 2 usages
}

import ...

@RestController
@RequestMapping("/alumnos")
public class AlumnoController {

    private final AlumnoService alumnoService; 7 usages

    public AlumnoController(AlumnoService alumnoService) { this.alumnoService = alumnoService; }

    @GetMapping
    public ResponseEntity<List<Alumno>> getAll() {
        List<Alumno> alumnos = alumnoService.obtenerTodosLosAlumnos();
        return ResponseEntity.ok(alumnos);
    }

    @GetMapping("/{idAlumno}")
    public ResponseEntity<Alumno> getById(@PathVariable Integer idAlumno) {
        return alumnoService.obtenerAlumnoPorId(idAlumno).Optional<Alumno>
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @PostMapping
    public ResponseEntity<Alumno> create(@RequestBody Alumno alumno) {
        Alumno nuevoAlumno = alumnoService.crearAlumno(alumno);
        return ResponseEntity.status(HttpStatus.CREATED).body(nuevoAlumno);
    }

    @PutMapping("/{idAlumno}")
    public ResponseEntity<Alumno> update(@PathVariable Integer idAlumno, @RequestBody Alumno datos) {
        try {
            Alumno alumnoActualizado = alumnoService.actualizarAlumno(idAlumno, datos);
            return ResponseEntity.ok(alumnoActualizado);
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @DeleteMapping("/{idAlumno}")
    public ResponseEntity<Void> delete(@PathVariable Integer idAlumno) {
        try {
            alumnoService.eliminarAlumno(idAlumno);
            return ResponseEntity.noContent().build();
        } catch (RuntimeException e) {
            return ResponseEntity.notFound().build();
        }
    }

    @GetMapping("/count")
    public ResponseEntity<Long> count() {
        long total = alumnoService.contarAlumnos();
        return ResponseEntity.ok(total);
    }
}
```

Dentro de nuestro AlumnoController hemos definido cada método para obtener todos los alumnos con getAll(), obtener por id con getById(), crear alumnos con create(), actualizarlos con update(), eliminarlos con delete() o contar el total de alumnos con count(). Además, las

notaciones de `@GetMapping`, `@PostMapping`, `@PutMapping`, etc. Sirven para que cuando llegue una petición HTTP como comprobaremos más adelante indique qué método en concreto tiene que ejecutarse.

Y, por último, en este apartado, tenemos también una clase `AlumnoService` que es donde ocurre toda la lógica de negocio. Para separar del Controller que se dedica a las peticiones se crea esta clase donde hacemos las validaciones, comprobaciones, transformaciones u operaciones.

```
package com.example.demo.Services;

import ...

@Service
public class AlumnoService {

    @Autowired
    private AlumnoRepository alumnoRepository;

    public List<Alumno> obtenerTodosLosAlumnos() { return alumnoRepository.findAll(); }

    public Optional<Alumno> obtenerAlumnoPorId(Integer idAlumno) { return alumnoRepository.findById(idAlumno); }

    public Alumno crearAlumno(Alumno alumno) { return alumnoRepository.save(alumno); }

    public Alumno actualizarAlumno(Integer idAlumno, Alumno alumnoDetalles) { // usage
        return alumnoRepository.findById(idAlumno)
            .map(Alumno alumno -> {
                alumno.setNombre(alumnoDetalles.getNombre());
                alumno.setApellidos(alumnoDetalles.getApellidos());
                alumno.setFechaNac(alumnoDetalles.getFechaNac());
                return alumnoRepository.save(alumno);
            })
            .orElseThrow(() -> new RuntimeException("Alumno no encontrado con id: " + idAlumno));
    }

    public void eliminarAlumno(Integer idAlumno) { // usage
        if (!alumnoRepository.existsById(idAlumno)) {
            throw new RuntimeException("Alumno no encontrado con id: " + idAlumno);
        }
        alumnoRepository.deleteById(idAlumno);
    }

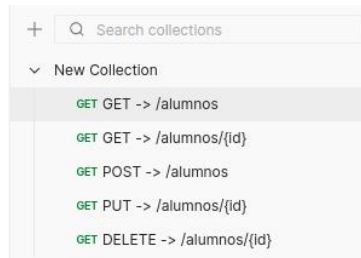
    public boolean existeAlumno(Integer idAlumno) { return alumnoRepository.existsById(idAlumno); }

    public long contarAlumnos() { return alumnoRepository.count(); }

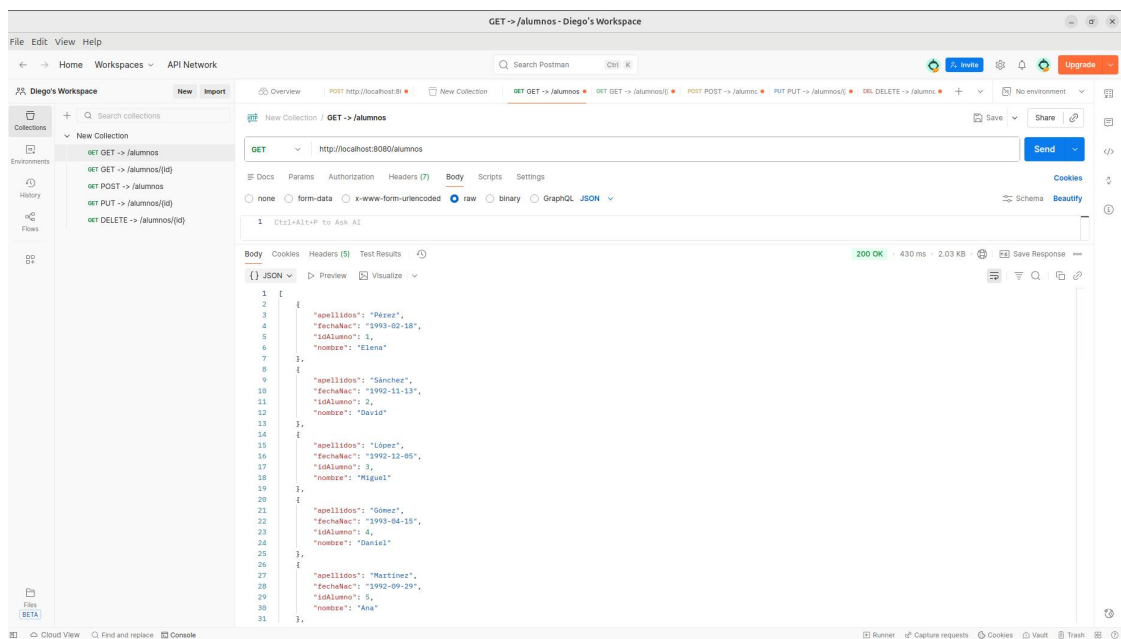
    public void eliminarTodosLosAlumnos() { alumnoRepository.deleteAll(); }
}
```

3. Implementación de Endpoints y pruebas

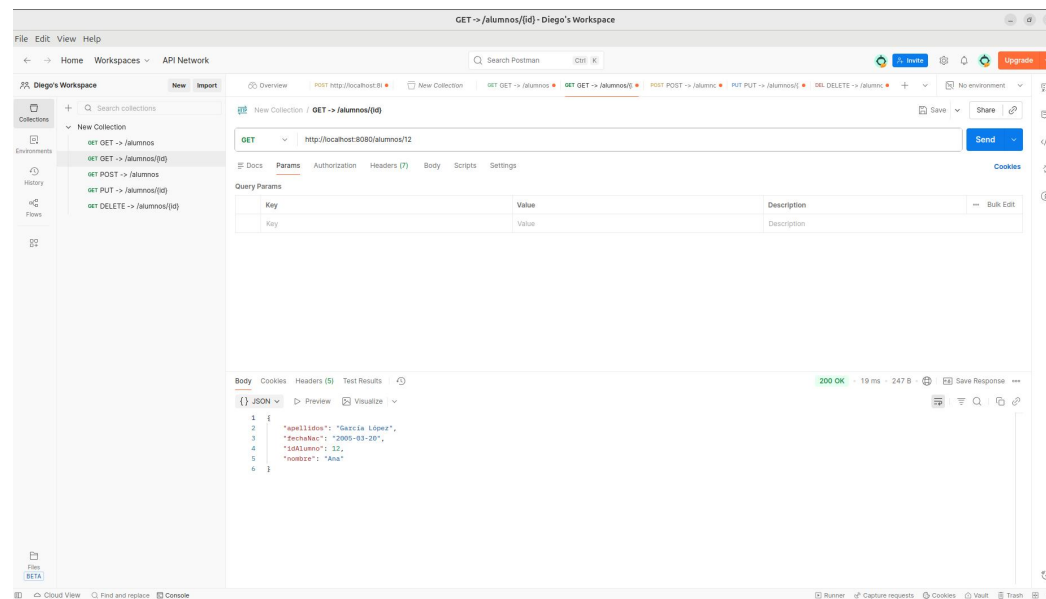
Finalmente, una vez hechas todas las nuevas clases y métodos que nos van a permitir migrar nuestro proyecto a Spring Boot, tenemos que dirigirnos a Postman donde haremos nuestras peticiones HTTP para comprobar que todo tiene sentido y que se ejecuta de forma correcta. En Postman vamos a crear una colección con los diferentes Endpoints que queremos ejecutar:



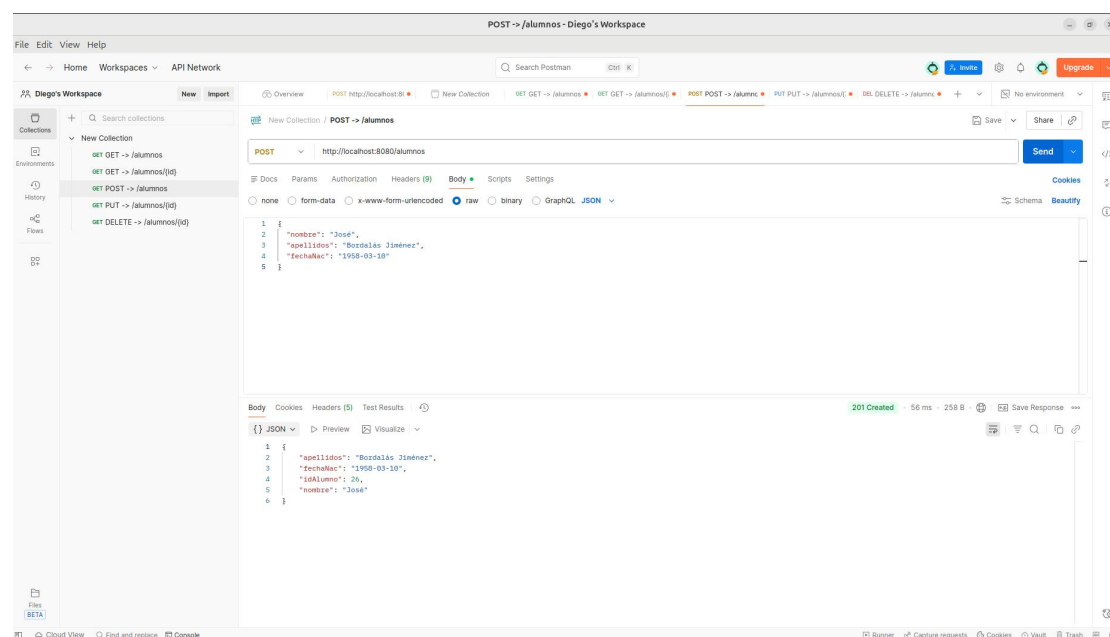
GET <http://localhost:8080/alumnos> nos va a permitir obtener todos los alumnos que haya registrados en la base de datos.



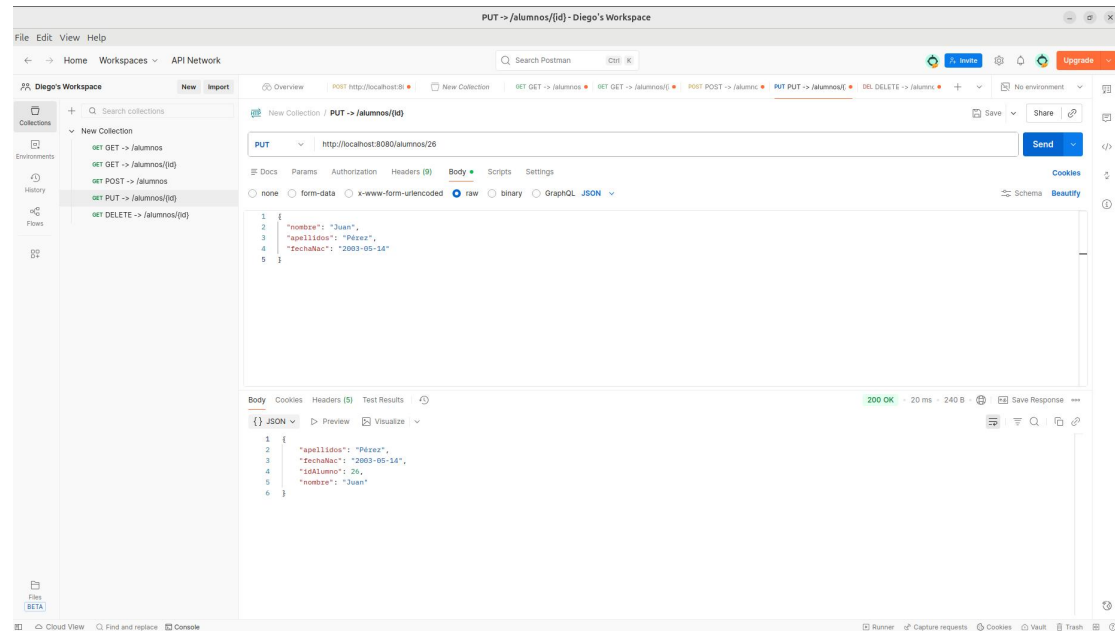
GET <http://localhost:8080/alumnos/1> va a permitir que dependiendo del número que indiquemos al final (corresponde al id de cada alumno) nos dará un alumno u otro.



POST <http://localhost:8080/alumnos> nos dejará añadir un nuevo alumno. Para ello, tenemos que ir al apartado de Body y señalar en el desplegable Raw. Una vez hecho esto, podemos poner en formato .JSON los campos que añadiremos al nuevo alumno. Cuando enviamos la petición, nos aparece abajo la nueva inserción de alumno.



PUT <http://localhost:8080/alumnos/1> sirve para actualizar uno de los alumnos, pero todos los campos, es decir, no podemos modificar solo uno. Para eso está otro Endpoint que es **PATCH**. Para hacer el PUT hacemos lo mismo que en POST, nos dirigimos a Body y Raw y ahí introducimos los campos en formato .JSON con nuestros cambios. Una vez hechos, saldrán actualizados abajo.



DELETE <http://localhost:8080/alumnos/1> este último sirve para eliminar directamente un alumno por id. Una vez enviada la petición, el alumno desaparece de la base de datos y si comprobamos haciendo un **GET** <http://localhost:8080/alumnos/1> nos dará un error 404 porque no habrá encontrado la petición.

